

Learnership LMS

Developer App and Views Guide

Team handout | September 2026

This guide explains how to claim an app, find the right files, and take a task from URL to tested page. It describes the current project conventions; it does not implement feature work for you.

Start here

- Claim one primary app and one task in the team task board before editing.
- Work in the existing app that owns the feature. All six project apps already exist; do not create duplicates.
- New and updated endpoints use Django function-based views. Do not add class-based views or mixins.
- Keep local development on SQLite. Docker and PostgreSQL are not part of the current setup.
- Enforce permissions on the server, scope returned data, and test both allowed and denied access.
- Keep changes focused on the task and coordinate cross-app model changes with each affected app owner.

1. Set up locally

Use Python 3.12 and Git. From PowerShell:

```
git clone https://github.com/Wise2man/Learnership-Management-System.git
cd Learnership-Management-System
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements/dev.txt
copy .env.example .env
python manage.py migrate
python manage.py create_demo_data
python manage.py runserver
```

Open <http://127.0.0.1:8000/>. The default database is SQLite; leave `DATABASE_URL` unset. Never commit `.env`, credentials, or real learner data. Demo accounts and their shared password are listed in [README.md](#).

Before a pull request, run `make check`. On Windows without Make, run the equivalent checks:

```
ruff check .
ruff format --check .
python manage.py check
python manage.py makemigrations --check --dry-run
pytest
```

2. Choose and claim an app

The app owner is the first contact for design questions and model/migration changes. Ownership coordinates work; it does not block collaboration.

App	Owns	Example work
accounts	Users, roles, profiles, registration and account administration	Profile editing, facilitator administration, role-related tests
courses	Courses, units, publishing and course discovery	Course forms, unit management, course lists

App	Owns	Example work
enrollments	Applications, decisions and student enrollments	Apply/withdraw workflows, approvals, enrollment lists
assessments	Tests, marks/results and feedback	Test management, marks entry, learner result pages
attendance	Class sessions and attendance records	Session management, marking attendance, reports
core	Shared home/dashboard pages and cross-app utilities	Role dashboards, shared access rules, shared page behavior

Confirm the app is unclaimed or ask its current owner. Record your claim and task in the team task board. Start from [docs/TASK_MAP.md](#); coordinate shared behavior with every affected owner.

3. Know the project layout

config/	Django settings and project-level URLs
<app>/	App models, forms, views, URLs and domain logic
models.py	Persistent data owned by this app
forms.py	Input fields and validation
views.py	Function-based HTTP request handlers
urls.py	Named routes for this app
services.py	Business operations and state changes
selectors.py	Reusable read/query operations
admin.py	Django admin registration
migrations/	Versioned database schema changes
templates/<app>/	HTML templates owned by the app
static/	Shared CSS, JavaScript and images
tests/test_<app>.py	App-focused tests (shared tests folder)

Not every task needs every module. Reuse the existing app structure and add files only when the feature needs them. Use `settings.AUTH_USER_MODEL` for user foreign keys. Never edit a migration that has already been merged; create a new migration instead.

4. Build a view from request to response

Follow this sequence for a page or action:

- Trace the route.** Find the app's `urls.py`, URL name, and current template/placeholder. Preserve the public URL unless the task calls for changing it. Put fixed routes before catch-all slug routes.
- Define the data needed.** Use the app's models and an existing selector where possible. Add a selector for a reusable read query; scope it to the current user, role, course, or assigned unit.
- Validate input.** Use a Django form for submitted data. Keep field and cross-field validation in `forms.py`, not scattered through the template.
- Enforce access.** Authenticate and authorize on the server before returning or changing data. Check object ownership as well as role. A hidden button is not an access check.
- Handle the request in a function view.** GET displays or reads; POST validates and performs a state change. Never mutate data on GET. Use the service layer for business operations that can be reused or span multiple writes.
- Connect the route.** Point the named URL pattern to the function view and keep names stable for templates and redirects.
- Render and redirect.** Put HTML in `templates/<app>/`. Reuse the base template and shared partials; show useful validation messages and an empty state. After a successful POST, redirect to a named URL.
- Test the behavior.** Cover a successful request, invalid input, anonymous access where relevant, denied roles, and object-level privacy. Add service/model tests for business rules.

Permission API in this starter

`core/permissions.py` currently exposes predicate functions: `can_manage_courses(user)`, `can_edit_course(user, course)`, `can_review_applications(user, course)`, and `can_teach_unit(user, unit)`. These return booleans; they are not view decorators. Use them as server-side authorization decisions and return/raise the appropriate denied response. If several apps need a reusable decorator, agree on its interface with the `core` owner and add tests there. Do not copy role logic into each view or assume a decorator exists.

Use Django's authentication and CSRF protections. State-changing forms must use POST and include `{% csrf_token %}`. Wrap related writes, such as approving an application and creating its enrollment, in `transaction.atomic()` inside the service.

5. Divide responsibilities cleanly

Concern	Put it here	Keep out of
HTTP method, request/response, messages, redirects	<code>views.py</code>	Models and templates
Field and form-level validation	<code>forms.py</code>	View condition sprawl
Business rules and coordinated writes	<code>services.py</code>	Templates and URL configuration
Reusable reads and queryset scoping	<code>selectors.py</code>	Repeated view queries
Persistent fields, relations and constraints	<code>models.py</code>	Views
Page structure and display states	<code>templates/<app>/</code>	Permission enforcement
Shared authorization predicates/decorators	<code>core/permissions.py</code>	Duplicated app-specific role checks

6. Database and cross-app changes

The app that owns a model owns its schema and migrations. Ask that owner before changing another app's model. Agree on the field/relationship contract first, then make the smallest migration needed. After model changes, run:

```
python manage.py makemigrations
python manage.py migrate
python manage.py makemigrations --check --dry-run
```

For approval, attendance, marks, or other multi-record changes, use a service function and an atomic transaction. Add uniqueness and validation constraints at the appropriate model/database boundary, with tests for edge cases.

7. Test and review checklist

- The endpoint is a function-based view; no new class-based views or mixins.
- Named URL, expected HTTP methods, redirects, and messages behave correctly.
- Authorization is checked server-side; object ownership and record privacy are tested.
- Invalid forms return errors without changing data; GET requests never mutate state.
- Querysets are scoped and list pages avoid unnecessary database queries.
- Templates include empty/error states, use named URLs, and work on narrow screens.
- Migrations are included when needed; secrets and personal data are excluded.
- `make check` passes, and the pull request names the task and requests review from the app owner.

8. Branch and collaboration flow

Create one feature branch per task from `develop`, for example `feature/LMS-203-course-crud`. Commit with the task ID, such as `LMS-203: add course management`. Push the feature branch and open a pull request into `develop`; do not push task work directly to `main` or `develop`. Refresh your branch from `develop` before review and rerun checks after resolving conflicts. See [docs/GIT_WORKFLOW.md](#) and [CONTRIBUTING.md](#) for the full rules.

Legacy starter note

Some existing endpoints still use class-based views and mixins. They are retained starter code, not the pattern for new work. Converting those endpoints is a separate, explicitly scoped task; do not silently rewrite them while implementing an unrelated feature.

Project references

- App claim and boundaries: [docs/APP_OWNERSHIP.md](#)
- Architecture and relationships: [docs/ARCHITECTURE.md](#)
- Feature ownership and statuses: [docs/TASK_MAP.md](#)
- Page implementation checklist: [docs/HOW_TO_BUILD_A_PAGE.md](#)
- Branch and migration rules: [docs/GIT_WORKFLOW.md](#)